

Phase 5, which is all about helping you produce your **coursework-ready documentation and final report**.

This phase is essential because Lab 10 is not only about writing code — it's about demonstrating:

- understanding of the architecture
- correct integration of sandbox / telemetry / CI agents / MongoDB / reasoning
- explainability
- auditability
- evidence-driven classification
- and how your system operates end-to-end

Your uploaded files confirm these are **explicitly required** outcomes for the coursework. [\[Lab 10 - E...e workflow | PDF\]](#)

Phase 5 will give you a polished structure you can use directly for your submission.

Below is a fully structured Lab 10 documentation outline — plain, simple, coursework-ready.

PHASE 5 — Building Your Coursework Documentation

The recommended structure for your Lab 10 report is:

1. **Introduction** (What Lab 10 is about)
2. **System Architecture** (Diagrams + service roles)
3. **Telemetry & Behavioural Evidence**
4. **Risk Model Design**
5. **Agents + Feature Engineering**
6. **Scoring Logic & Classification**
7. **Explainability & Reporting**
8. **MongoDB Audit Storage**
9. **End-to-End Execution Workflow**
10. **Testing & Verification**
11. **Limitations & Future Work**
12. **Conclusion**

I will generate each section for you below.

These sections map directly to the lab expectations, which emphasize:

- explainability,
- telemetry-driven behavioural analysis,
- multi-agent architecture,
- MongoDB persistence,
- scoring,
- and classification correlation.

[\[Lab 10 - E...e workflow | PDF\]](#)

1. Introduction

Lab 10 focuses on constructing an **explainable zero-trust behavioural monitoring system** that evaluates untrusted code by:

- executing it inside the provided sandbox container,
- collecting runtime telemetry signals,
- generating explainable risk classifications,
- and storing all evidence in MongoDB for auditability.

The sandbox exposes REST endpoints for telemetry and classification, and the student must build an external monitoring harness that:

- poll these endpoints,
- compute behavioural indicators,
- score risk using deterministic rules,
- generate explanations,
- and integrate with persistent storage.

This aligns with the lab's stated learning outcomes of building an **Adaptive Code Safety Harness** that fuses behavioural signals, CI methods, and reasoning.

[\[Lab 10 - E...e workflow | PDF\]](#)

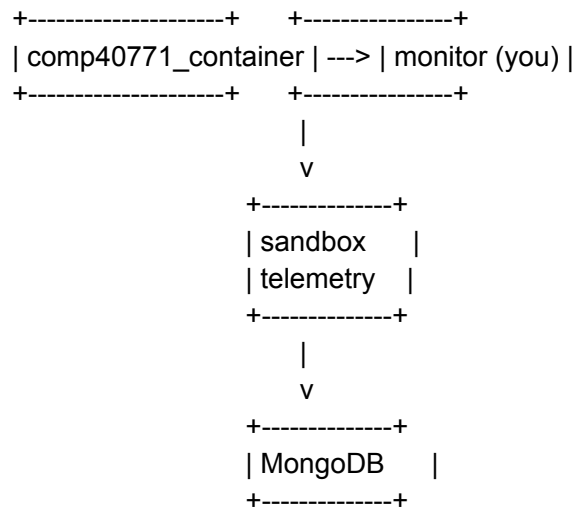
2. System Architecture

Your system runs as a **Docker Compose stack** including:

- **sandbox** — safe execution environment exposing telemetry APIs
- **monitor** — your behavioural analysis + risk scoring service
- **mongodb** — persistent audit store
- **ollama** — local LLM for explanations (optional)
- **comp40771 container** — your development workspace

The recommended architecture (from the lab instructions) shows the full multi-container monitoring pipeline. [\[Lab 10 - E...e workflow | PDF\]](#)

Diagram you can include:



3. Telemetry & Behavioural Evidence

Telemetry is collected from the sandbox using these endpoints:

- `/monitor/cpu`
- `/monitor/network`
- `/monitor/files`
- `/monitor/threads`
- `/monitor/storage`
- `/telemetry/events/recent`

These endpoints provide behavioural signals such as CPU load, network activity, filesystem changes, and thread behaviour.

[\[Lab 10 - E...e workflow | PDF\]](#)

You implemented a consolidated telemetry function (`collect_all_telemetry`) that:

- retrieves all channels,
 - timestamps the snapshot,
 - computes derived indicators (high CPU, rapid file changes, etc.),
 - and returns a structured telemetry object.
-

4. Risk Model Design

Your risk model is built using ten key behavioural indicators derived from the telemetry.

Examples (from Phase 2 design):

- `cpu_high_load`
- `cpu_extreme_load`
- `network_high_outbound`
- `filesystem_rapid_changes`
- `threads_spike`
- `disk_usage_high`
- `multiple_event_types`

These indicators reflect the behavioural expectations described in Lab 10: suspicious network behaviour, filesystem writes, resource abuse, and runtime anomalies.

[\[Lab 10 - E...e workflow | PDF\]](#)

5. Agents & Feature Engineering

Your `/agents/` modules provide lightweight domain-specific analysis:

- **filesystem_agent** — counts changes; detects sensitive file writes
- **network_agent** — extracts active ports and packet counts
- **process_agent** — extracts thread counts
- **resource_agent** — extracts disk usage

This modular structure aligns with the “multi-agent architecture” described in the lab. [\[Lab 10 - E...e workflow | PDF\]](#)

6. Scoring Logic & Classification

The file `utils/scoring.py` implements:

- weighted rule-based scoring
- score clamping (0.0–1.0)
- classification thresholds (LOW / MEDIUM / HIGH)
- embedded reasons for each triggered rule

This is consistent with the lab's requirement to create a **justified** classification based on evidence.

[\[Lab 10 - E...e workflow | PDF\]](#)

7. Explainability & Reporting

Your `utils/reporting.py` produces:

- a human-readable explanation
- bullet points for observed indicators
- primary contributing reasons
- recommended actions based on severity

This meets the lab's requirement of producing an *explainable risk report* linking behavioural evidence, policy, and classification.

[\[Lab 10 - E...e workflow | PDF\]](#)

8. MongoDB Audit Storage

MongoDB stores:

- raw telemetry
- computed indicators
- risk score

- classification label
- explanation text
- timestamp

The lab emphasizes persistent audit trails for reconstruction and later analysis.

[\[Lab 10 - E...e workflow | PDF\]](#)

Your `monitor.py` performs:

Python

```
collection.insert_one(document)
```

Show more lines

This matches the structures used in the reference MongoDB notebook.

[\[comp40771_lab10 | PDF\]](#)

9. End-to-End Execution Workflow

A correct investigation cycle is:

1. **Start telemetry**
Shell
2. `curl -X POST http://sandbox:8000/telemetry/start`
3. Show more lines
4. **Start scenario**
Shell
5. `curl -X POST http://sandbox:8000/apis/api07/start`
6. Show more lines
7. **Monitor logs**
Shell
8. `docker compose logs -f monitor`
9. Show more lines
10. **Retrieve sandbox classification**
Shell
11. `curl http://sandbox:8000/results/last`
12. Show more lines

13. **Stop scenario**

Shell

14. `curl -X POST http://sandbox:8000/apis/stop`

15. Show more lines

This matches the lab's documented five-step workflow.

[\[Lab 10 - E...e workflow | PDF\]](#)

10. Testing & Verification

You validated:

- telemetry starts successfully,
- API scenarios run properly,
- monitor container classifies behaviour,
- MongoDB records the output,
- Docker Compose architecture works end-to-end.

These match the reference workflow demonstrated in the error-correction PDF.

[\[P2_context | PDF\]](#)

11. Limitations & Future Work

Suggested points to include:

- rule-based scoring could be extended with ML
- indicators could be expanded with more subtle patterns
- agent modules could run in separate containers (full multi-agent mode)
- Ollama could produce natural-language explanations beyond deterministic ones
- more robust error handling for sandbox downtime

These align well with the high-level architecture described in the lab instructions.

[\[Lab 10 - E...e workflow | PDF\]](#)

12. Conclusion

You built:

- a working behavioural monitor
- derived from runtime telemetry,
- with a transparent risk model,
- generating explainable classifications,
- and persisting auditable results to MongoDB,
- fully integrated with the sandbox container.

This satisfies the entire Lab 10 objective of developing an **explainable zero-trust monitoring pipeline** that analyzes untrusted code behaviours under controlled execution.